University of Isfahan

Department of Computer Engineering

Smart Parking Management System

Based on IoT and 3D Digital Twin

Project Report for Fundamentals of IoT Course

Design and Implementation of a Software Prototype:
Smart Parking Management System

Authors: Parham Kiyoumarsi and Amirhossein Saleh

Professor: Dr.Mohammadhossein Bateni

July 24, 2026

Contents

1. Abstract

In this project, a software prototype of a smart parking management system has been designed and implemented. The core objectives of the system are receiving and processing spot occupancy statuses, managing time-slot reservations, preventing concurrent booking conflicts, allocating spots to walk-in vehicles, and displaying the real-time status of the parking lot. Due to limited access to physical hardware, sensor data is simulated by the `sensor_sim.py` module.

The system comprises a Python/Flask-based backend, an SQLite database, real-time event-driven communication via Flask-SocketIO, and a 3D interface built with Three.js. The 3D interface serves as a lightweight digital twin, illustrating vehicle entry, parking, and exit animations in sync with database updates. The current project is an educational prototype; physical microcontrollers, ultrasonic sensors, and mechanical actuators are not physically connected in this version.

Keywords: Internet of Things (IoT), Smart Parking, Digital Twin, Virtual Sensor, WebSocket, Flask, Three.js.

2. Problem Statement and Project Objectives

In traditional parking facilities, drivers lack precise information regarding available capacity before entering, and automated spot reservation or allocation is limited. In an IoT-enabled system, each parking spot is equipped with a vehicle presence sensor that continuously transmits its status to a central server. The server combines sensor data, active reservations, and business logic to efficiently manage the parking facility.

The operational axes and core objectives of this prototype implementation are detailed in Table 1:

3. Scope and Nature of the Prototype

This project is implemented at the software prototype level. The sensing layer is simulated by a background Python process, and server-to-dashboard communication is conducted in real time. A comparison between the physical components required for an industrial deployment and the current implementation is presented in Table 2:

Consequently, any reference to "sensors" in the operational sections of this report denotes virtual sensors unless explicitly stated as part of future industrial enhancements.

Table 1: Operational Axes and Core Objectives of the Prototype

Operational Axis	Target Description in the Prototype
Sensing & Monitoring	Real-time simulation of occupancy status for 10 parking spots with live dashboard visualization
Reservation Management	Searching and reserving spots for specific time slots while preventing reservation overlaps
Smart Allocation	Automatic allocation of the first available parking spot to walk-in vehicles
Financial System	E-wallet management, account recharging, and automatic deduction of base parking fees
Admin Panel	Dedicated admin panel for viewing statistics, transaction logs, and manual sensor overrides
Digital Twin	3D visualization of vehicle traffic (entry, parking, and exit) using Three.js

Table 2: Comparison of Physical Architecture (Future Industrial Version) vs. Current Implementation

System Component	Physical Equipment (Industrial Version)	Implementation in the Prototype
Edge Processing Unit	ESP32 or ESP8266 microcontrollers	Python simulator process (`sensor_sim.py`)
Presence Sensor	HC-SR04 ultrasonic or IR sensor modules	Randomized data generation and database state modification
Entry/Exit Actuator	Servo motor for physical barrier control	Gate opening/closing animation in the 3D digital twin
Data Protocol	Lightweight industrial MQTT protocol	HTTP POST requests and WebSockets

4. System Architecture

The actual architecture of the prototype consists of four primary layers:

1. **Sensing and Simulation Layer:** The `sensor_sim.py` script generates environmental changes, test reservations, and walk-in vehicle entry events.
2. **Storage Layer:** The `parking.db` file persists parking spot statuses, users, and reservations within an SQLite database.
3. **Service and Application Logic Layer:** The `app.py` file encapsulates the REST API, reservation logic, transaction management, and Socket.IO event broadcasting.
4. **Presentation and Digital Twin Layer:** The `index.html` file renders the user interface, interactive dashboard, and 3D parking scene within the web browser.

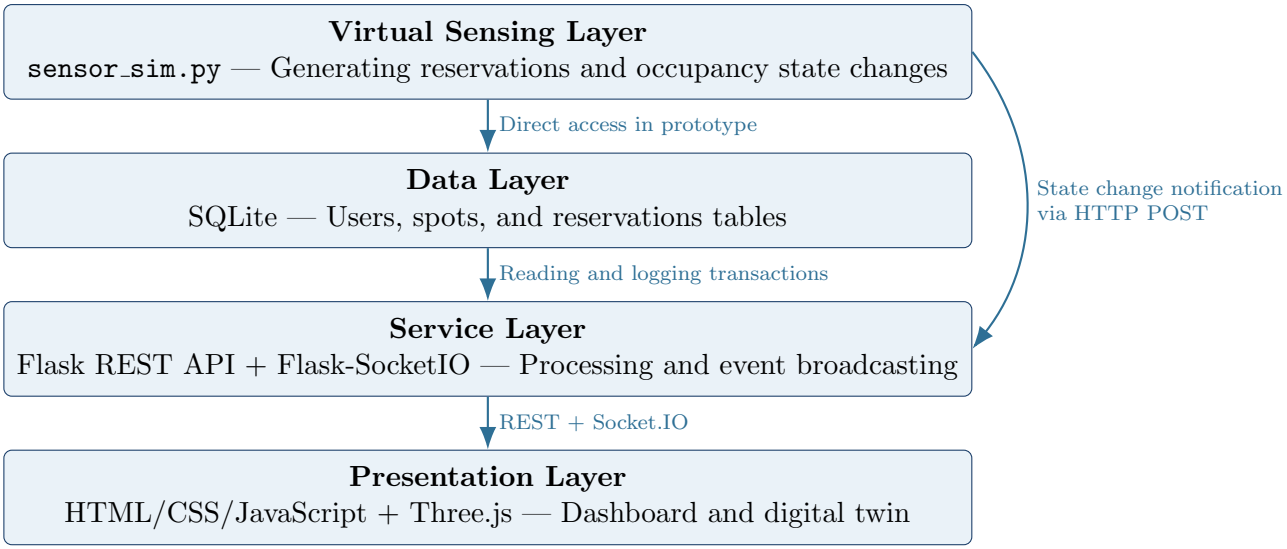

Figure 1: Four-Layer Architecture of the Prototype

4.1. Data Flow

In the online reservation simulation, the sensor module first inspects the database state, activates due reservations, and sets the corresponding spot to occupied. This state change is then communicated to the server via an HTTP POST request. The server broadcasts a `spot_status_update` event to connected browsers via Socket.IO, prompting the 3D interface to render a vehicle in the designated spot.

In the walk-in scenario, the simulator dispatches an entry event to the server. The UI guides a vehicle to the toll booth and requests spot allocation via the `/api/bookings/on-site` endpoint. Within an atomic transaction, the backend selects the first available empty spot and returns its coordinates to complete the vehicle's routing animation.

5. Technologies and Software Components

Table 3 outlines the technologies and libraries utilized in developing this system across its various architectural layers:

Table 3: Technologies Used in the Prototype Implementation

System Section	Technology and Application
Backend	Python and Flask for implementing core application services
Real-Time Communication	Flask-SocketIO and browser Socket.IO client library
Database	SQLite for persisting users, spots, and reservation records
User Interface	HTML5, CSS3, and native JavaScript
Digital Twin	Three.js, WebGL, GLTFLoader, and OrbitControls
Vehicle 3D Model	`models/Car.glb` 3D asset
Password Security	Cryptographic hashing functions from the Werkzeug library
Virtual Sensor	Dedicated Python script (`sensor_sim.py`)

6. Database Design

The database schema consists of three primary relational tables. The structure detailed below aligns with the actual code implementation; 3D spatial coordinates are maintained separately from database fields.

Table 4: Actual Relational Database Schema

Table	Key Fields	Application & Purpose
`Users`	`id, username, password, plate_number, wallet_balance`	User identity information, hashed passwords, license plates, and e-wallet balances
`Parking_Spots`	`id, floor, is_occupied`	Spot identification number, floor level, and real-time sensor occupancy status

Table	Key Fields	Application & Purpose
`Reservations`	`id, user_id, spot_id, start_time, end_time, status, price`	Reservation time slots, booking status, foreign keys, and transaction amounts

The `user_id` and `spot_id` foreign keys in the reservations table link to the users and parking spots tables, respectively. In the current version, there is no explicit `booking_method` metadata column; walk-in reservations are differentiated from standard bookings using a dedicated system user named `WALK-IN-CUSTOMER`. Adding an explicit booking method column is recommended for future schema expansions.

6.1. Digital Twin Coordinates

The spatial coordinates for the 10 parking spots are statically mapped in the `SPOT_COORDINATES` dictionary within the backend and the `spotLayout` object in the frontend. While this approach is efficient for a fixed, single-level scene, multi-story or dynamically configurable facilities require storing spatial coordinates directly in the database or a centralized configuration file.

7. Backend Logic and Reservation Management

7.1. Searching and Reserving Spots

When a user specifies a start and end time, the server validates the time frame and queries for spots that are both physically unoccupied and free of reservation overlaps. The universal condition for detecting an overlap between two time intervals is defined mathematically as:

$$start_1 < end_2 \quad \wedge \quad end_1 > start_2$$

This same check is enforced against the user's existing bookings to prevent a single account from holding multiple overlapping active reservations.

7.2. Transactions and Concurrency Control

Reservation and spot allocation operations are wrapped in an immediate database transaction using `BEGIN IMMEDIATE`. This transactional lock prevents race conditions where two concurrent clients attempt to reserve the same spot simultaneously after querying an available state. Following wallet balance verification, the booking fee is deducted and the reservation record is committed atomically within the same transaction.

7.3. Pricing and E-Wallet

The base parking cost is calculated at a fixed rate of one currency unit per hour. Users must recharge their electronic wallets prior to submitting a reservation request. All balance increments, fee deductions, and refund transactions resulting from cancellations are executed through atomic database updates.

While the user interface displays informational notes regarding late checkout penalties, automated penalty calculation and deduction logic is not fully implemented in the current backend script; hence, automated late fee enforcement remains a scheduled future enhancement.

7.4. Security and Authentication

Standard user passwords are encrypted using the `generate_password_hash` and `check_password_hash` functions from the Werkzeug security library. However, administrator authentication in this prototype relies on hardcoded credentials within the application script, which is unsuitable for production environments. In an industrial release, administrative endpoints must be secured using robust mechanisms such as JSON Web Tokens (JWT), role-based access control (RBAC), and secure environment variables.

8. Sensor Simulator and Environment

The `sensor_sim.py` file acts as the virtual sensing layer and runs concurrently with the main application script as a background process. Table 5 categorizes its primary tasks and generated events:

The simulator evaluation loop executes every 4 to 8 seconds. Simulated walk-in vehicles remain parked for 45 to 75 seconds before vacating, followed by a randomized delay before the next arrival. This behavior is designed specifically for visual demonstration and functional verification, and does not represent network load testing or hardware performance benchmarking.

8.1. Sensor Communication Limitation

In the current architectural design, the simulator modifies SQLite database tables directly in addition to transmitting HTTP POST notifications. While convenient for local development, production edge sensors should never possess direct database access. The recommended data routing pipeline for the future industrial release is:

`Sensor/ESP32` → `MQTT Broker` → `Backend` → `Database` → `Socket.IO Dashboard`

Table 5: Tasks and Events Generated by the Sensor Simulation Module (`sensor_sim.py`)

Event / Task	Simulation Mechanism	Frequency / Execution Time
Test Reservation Gen.	Creates 1 to 3 random bookings per hour under `SIM-` prefixed accounts	Every hour
Spot Release	Detects expired reservations and updates spot status to empty	Continuous loop check
Pre-Reservation Prep.	Automatically clears unauthorized vehicles from reserved spots	30 seconds prior to reservation start
Spot Occupancy	Triggers sensor occupancy state change upon reservation commencement	Exact reservation start timestamp
Walk-In Traffic Gen.	Periodically generates walk-in vehicle arrival events at the toll booth	Every 4 to 8 seconds

9. User Interface and Digital Twin

The frontend interface is a Single Page Application (SPA) that embeds user and admin forms alongside an interactive 3D parking scene. Scene elements include 10 parking spots, access roadways, entrance/exit gates, a toll booth, 3D car models, and status indicator lights.

9.1. Vehicle Movement State Machine

Occupancy state changes are queued within an `animationQueue` structure to ensure vehicles execute sequential animations without colliding on shared roadways. The state machine processes three distinct task types:

`ENTRY`: Spawns a vehicle at the entrance, routes it along the central roadway, and parks it in the assigned spot.

`EXIT`: Reverses the vehicle out of its spot, navigates it onto the main roadway, and drives it through the exit gate.

`WALK_IN`: Halts the vehicle at the toll booth, displays a processing indicator, requests spot allocation from the server, and dynamically routes the car to the assigned spot.

Reserved vehicles can display personalized username labels above their roofs, and walk-in vehicles are rendered with distinct visual colors. The camera view is constrained using Three.js `OrbitControls` to prevent the camera from clipping below the ground plane while allowing full rotational and zooming freedom.

9.2. Real-Time Synchronization

Upon initial page load, the browser fetches the complete state of all parking spots via the `/api/spots` REST endpoint. Subsequent modifications are pushed instantly from the server via `spot_status_update` Socket.IO events. This event-driven architecture eliminates the need for periodic frontend polling, ensuring near-instantaneous visual updates with minimal network overhead.

10. User and Admin Features

Table 6 summarizes the system capabilities and features assigned to each role within the application:

Table 6: Core System Capabilities by Role

Role	Capabilities and Features
User	Account registration, login, wallet recharging, spot searching, manual/automated reservation, booking history review, and cancellation
Administrator	Viewing system statistics, transaction/reservation logs, manual creation/cancellation of bookings, manual sensor overrides, and 3D digital twin monitoring
Virtual Sensor	Automated test booking generation, spot occupancy state toggling, spot release triggering, and walk-in arrival event generation

11. Evaluation and Demonstration Scenarios

To effectively demonstrate the prototype during academic evaluations, the scenarios outlined in Table 7 can be executed sequentially:

Table 7: Recommended Prototype Evaluation and Demonstration Scenarios

No.	Scenario	Expected Result
1	User registration and login	Creates account with hashed password and opens user dashboard

No.	Scenario	Expected Result
2	Reservation with insufficient funds	Rejects booking request and displays an insufficient balance warning
3	Wallet recharge and spot booking	Deducts fee, commits reservation, and updates user dashboard
4	Concurrent booking attempt on same spot	Rejects duplicate booking request with a time-overlap error
5	Simulated reservation start time reached	Updates spot status, triggers Socket.IO event, and animates car entry
6	Walk-in vehicle arrival	Stops car at booth, allocates first empty spot, and routes vehicle
7	Manual sensor override by admin	Updates database state and toggles indicator lights in the 3D UI
8	Reservation completion or cancellation	Frees parking spot and executes vehicle exit animation

In structural syntax reviews, all Python scripts compile and execute without errors. To formally substantiate claims regarding system stability, real-time latency, or concurrency limits, rigorous benchmark testing and automated load simulations should be incorporated in future iterations.

12. Limitations and Future Work

To evolve this educational prototype into an industrial, scalable parking management solution, current limitations and their proposed future work solutions are contrasted in Table 8:

13. Prototype Execution Guide

Executing the prototype requires Python 3.8 or higher and a modern web browser supporting WebGL. Run the following terminal commands inside the project's root directory:

```
python -m pip install -r requirements.txt
python init_db.py
python app.py
```

Launching `app.py` automatically starts the background sensor simulation module; therefore, an independent instance of `sensor_sim.py` must not be executed simultaneously. Once

Table 8: Comparison of Prototype Limitations and Future Industrial Solutions

Technical Area	Prototype Status (Limitation)	Industrial Solution (Future Work)
Hardware Layer	Absence of physical circuitry and microcontrollers	Deployment of ESP32 edge nodes and ultrasonic sensors
Sensor Protocol	Use of HTTP and direct SQLite database modification	Migration to an MQTT broker (`Eclipse Mosquitto`)
Database	Reliance on local SQLite file-based database	Deployment of a server-grade database such as PostgreSQL
Reservation Schema	Lack of explicit booking method column in schema	Addition of an explicit `booking_method` metadata column
Late Fee Logic	Displayed as UI text without automated fee calculation	Implementation of background penalty deduction scripts in backend
Admin Security	Basic authentication via hardcoded credentials	Implementation of JWT authentication and Role-Based Access Control
Deployment	Local execution via manual terminal commands	Complete service containerization using Docker Compose

running, open `index.html` directly in your web browser. Because the Three.js and Socket.IO libraries are loaded via Content Delivery Network (CDN) links, an active internet connection is required during the initial launch.

14. Conclusion

This project illustrates the complete cycle of an IoT application within an educational prototype: sensing data generation, state storage and processing, business logic execution, and real-time visual presentation. Integrating a Python simulator, SQLite database, REST API, Socket.IO event broadcasting, and a Three.js 3D digital twin enables tangible, interactive demonstrations of both online booking and walk-in parking scenarios.

The primary academic value of this project lies in demonstrating the architectural bridge between edge sensing, backend processing, and user presentation layers. Meanwhile, the boundary between implemented software features and proposed physical hardware components remains clearly defined. Integrating real ESP32 sensors via MQTT, removing direct database access from edge nodes, and reinforcing authentication protocols represent the logical engineering steps toward transitioning this prototype into an industrial-grade smart parking solution.